



資料結構與演算法 I

(Data Structure and Algorithms I)

2025 fall

2025/09/19

Analysis of Algorithms

- ✱ For most situations, it is adequate to be able to make a statement like

$$\text{✱ } c_1n^2 \leq t_p(n) \leq c_2n^2$$

$$\text{or } t_Q(n,m) = c_1n + c_2m$$

- ✱ c_1, c_2 are nonnegative constants

- ✱ 如果是 $c_1n^2 + c_2n$ & c_3n 比大小??

- ✱ When $n \rightarrow$ large value, c_3n will be faster than $c_1n^2 + c_2n$

- ✱ When $n \rightarrow$ small value, either program could be faster, depending on c_1, c_2 and c_3

If $c_1=1, c_2=2$ and $c_3=100$
 $c_1n^2 + c_2n \leq c_3n$ for $n \leq 98$

If $c_1=1, c_2=2$ and $c_3=100$
 $c_1n^2 + c_2n > c_3n$ for $n > 98$

If $c_1=1, c_2=2$ and $c_3=1000$
 $c_1n^2 + c_2n \leq c_3n$ for $n \leq 998$

- ✱ No matter what the value of c_1 , c_2 and c_3 , there will be an n beyond which the program with complexity c_3n will be faster than $c_1n^2+c_2n$

- ✱ 不管 c_1 , c_2 and c_3 的數值為何，一定存在一個足夠大的 n ，使得 c_3n 的程式效率比 $c_1n^2+c_2n$ 快

- ✱ The value of n will be called the *break-even point*

- ✱ *The program have to be run on a computer for determination of the break-even point.*

- ✱ To know there is a break-event point, it is adequate to know that one program has complexity $c_1n^2+c_2n$ and the other c_3n for some constants: c_1 , c_2 and c_3 .

- ✱ That is little advantage in determining the exact value

次數	演算法A($2n+3$)	演算法A'($2n$)	演算法B($3n+1$)	演算法B'($3n$)
$n=1$	5	2	4	3
$n=2$	7	4	7	6
$n=3$	9	6	10	9
$n=10$	23	20	31	30
$N=100$	203	200	301	300

$n=1$ 演算法A效率不如演算法B

$n=2$ 效率相同

$n>2$ 演算法A效率開始優於演算法B

結論：演算法A總體效率比演算法B好

函式的漸進增長

給定兩個函式 $f(n)$ 跟 $g(n)$ ，如果存在一個整數 N ，且所有 $n>N$ ， $f(n)$ 總是比 $g(n)$ 大，則 $f(n)$ 的增長會逐漸快於 $g(n)$

隨著 n 的增加，演算法次數的常數項 $+3$ or $+1$ 其實都不影響最終的演算法，因此可以忽略這些加法常數

次數	演算法C($4n+8$)	演算法C'(n)	演算法D($2n^2+1$)	演算法D'(n^2)
n=1	12	1	3	1
n=2	16	2	9	4
n=3	20	3	19	9
n=10	48	10	201	100
n=100	408	100	20001	10000
n=1000	4008	1000	2000001	1000000

當 $n \leq 3$ ，演算法C比D差

當 $n > 3$ ，演算法C優勢就出現了

去掉後面常數也沒差

就算去掉與 n 相乘的常數沒沒啥差

演算法C'次數隨 n 增長，然遠小於演算法D'

與最高次項相乘的常數並不重要

次數	演算法 $E(2n^2+3n+1)$	演算法 $E'(n^2)$	演算法F $(2n^3+3n+1)$	演算法 $F'(n^3)$
$n=1$	6	1	6	1
$n=2$	15	4	23	8
$n=3$	28	9	64	27
$n=10$	231	100	2031	1000
$n=100$	20301	10000	2000301	1000000

當 $n=1$ ，演算法E跟F結果相同

當 $n>1$ ，演算法E表現出較佳的優勢

隨著 n 的增加，差異越來越明顯

最高次項的指數較大，則函式隨著 n 的遞增，次數就會遞增的特別快

次數	演算法G($2n^2$)	演算法H ($3n+1$)	演算法I($2n^2+3n+1$)
n=1	2	4	6
n=2	8	7	15
N=5	50	16	66
n=10	200	31	231
n=100	20000	301	20301
n=1000	2000000	3001	2003001
n=10000	200000000	30001	200030001
n=100000	20000000000	300001	20000300001
n=1000000	2000000000000	3000001	2000003000001

當n越來越大， $3n+1$ 已經無法跟 $2n^2$ 相比
 且n越來越大，演算法G跟I根本就相似

因此要判斷一個演算法的效率，函式中的常數與其他次要項目是可以忽略的，要關注的只有主項(最高項次)的階數

Big O''oh''

這裡的 $f(n)$ 是指演算法的
總執行次數

✱ Definition [Big “O”]:

✱ read as “f of n is big oh of g of n”

✱ $f(n) = O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n, n \geq n_0$.

✱ Upper bound on $f(n)$

✱ $O(1)$: a constant.

✱ $f(n) = O(g(n))$ to be informative, $g(n)$ should be as small a function of n as one can come up with for which $f(n) = O(g(n))$.

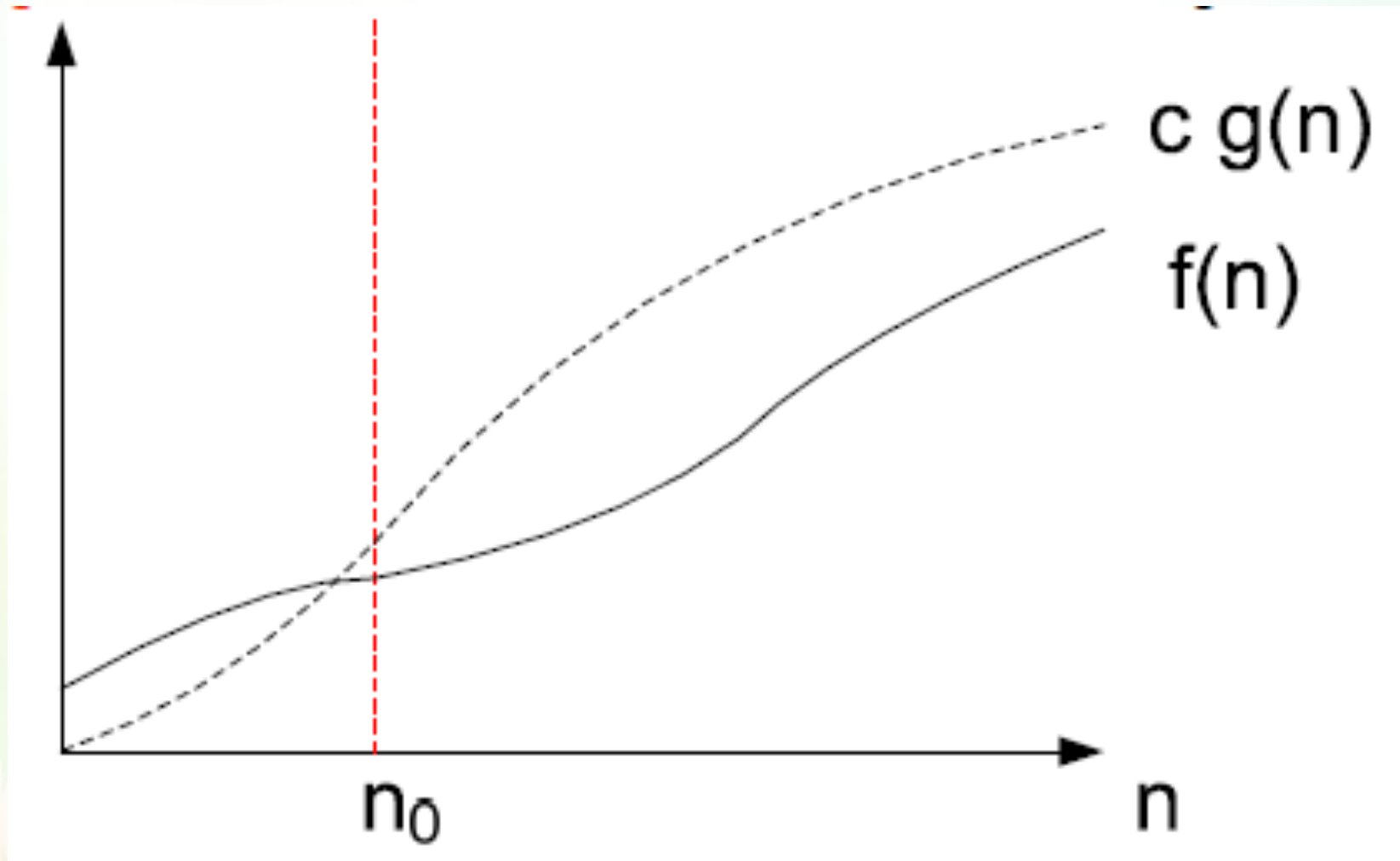
✱ = as **is** not as **equal**

Iff: if and only if

若且為若

在，並且僅僅在這些條件成立的時候

Big O



Example of Big “O”

- * $3n+2 = O(g(n))$

- * $f(n)=3n+2$, but how to select $g(n)$?

- * Because $cg(n)$ will be the upper bound

- * Of course, you could assign $g(n)=n^2$ (even though this latter statement is correct...)

- * But it will not be the upper bound

- * $g(n)=n \rightarrow$ calculate $3n+2 \leq cn$

- * try to assign c as the number which is bigger than 3

- * $c=4 \rightarrow$ calculate $3n+2 \leq 4n \rightarrow n_0=2$

- * $3n+2 \leq 4n$ for all $n \geq 2$.

Example

- ✱ $10n^2+4n+2 = O(g(n))$

- ✱ First, please assign the $g(n)$

- ✱ It does not say anything about how good this bound is

- ✱ $g(n)$ should be as small a function of n as one can come up with for which $f(n)=O(g(n))$

- ✱ $g(n)=n^2$

- ✱ How about c ? just bigger than 10 is ok! $\rightarrow c=11$

- ✱ $10n^2+4n+2 \leq 11n^2 \rightarrow n_0 \geq 5$

Common of Big O

- ✱ $12, O(1)$: constant 常數階
 - ✱ 表示 n 的大小與執行時間無關，注意沒有 $O(3)$, $O(10)$
- ✱ $5 \log n + 20, O(\log n)$: logarithmic 對數階
- ✱ $3n+2, O(n)$: linear 線性階
- ✱ $2n+3n \log n + 19, O(n \log n)$: log linear 對數線性階
- ✱ $6n^2+2n+1, O(n^2)$: quadratic 平方階
- ✱ $8n^3+3n^2+n+4, O(n^3)$: cubic 立方階
- ✱ $2^n+7, O(2^n)$: exponential 指數階
- ✱ $5n!+2, O(n!)$: factorial 階層階

$O(1) < O(\log n) < O(n) < O(n \log n) <$

$O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

Omega Ω

* Definition [Omega]

* Read as “f of n is omega of g of n”

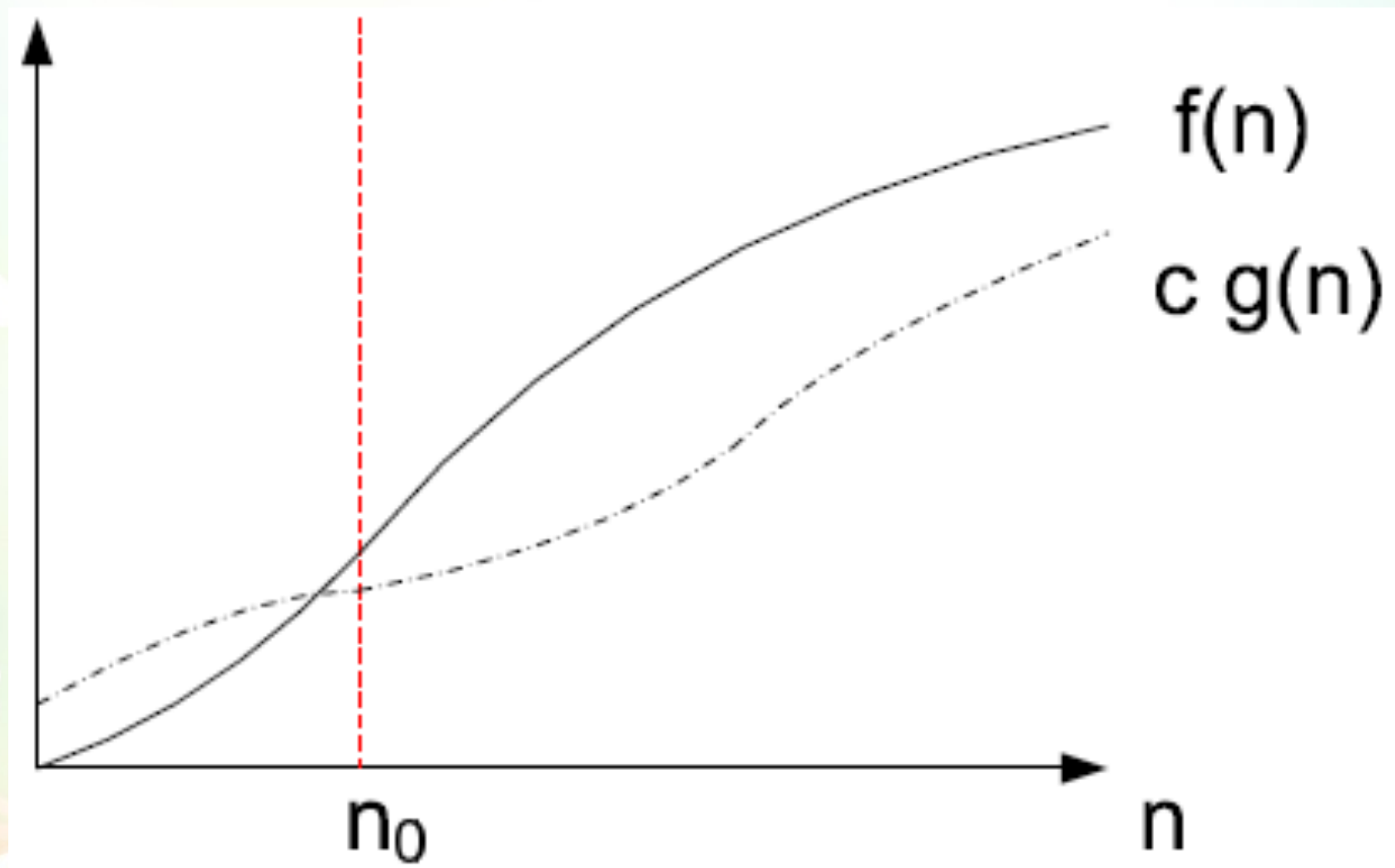
* $f(n) = \Omega(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n, n \geq n_0$

* The function $g(n)$ is only a **lower bound** on $f(n)$.

* Note: $O(g(n))$, $g(n)$ is the **upper bound** on $f(n)$

* $f(n) = \Omega(g(n))$ to be informative, $g(n)$ should be as large a function of n as one can come up with for which $f(n) = \Omega(g(n))$ is true.

Omega Ω



Examples of Omega

- * $3n+2 = \Omega(n)$

- * $f(n)=3n+2, g(n)=n$

- * $c=3$ (note: for Big O, c should be 4)

- * $3n+2 \geq 3n \rightarrow$ for all $n \geq 1$.

- * In fact, n could be 0, but the definition of Ω requires an $n_0 > 0$. That is the reason that $n \geq 1$.

- * $10n^2+4n+2 = \Omega(n^2)$

- * $f(n)= 10n^2+4n+2, g(n)= n^2, c=1, n_0=1$

- * n_0 的計算請用代入法

- * $10n^2+4n+2 \geq n^2$ for all $n \geq 1$.

✱ $6 \cdot 2^n + n^2 + 2 = \Omega(2^n)$

✱ $f(n) = 6 \cdot 2^n + n^2 + 2, g(n) = 2^n, c=1, n_0=1$

✱ n_0 的計算請用代入法

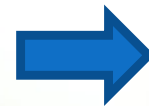
✱ $6 \cdot 2^n + n^2 + 2 = 2^n, \text{ for all } n \geq 1.$

✱ Is the following formula correct?

✱ $f(n) = 6 \cdot 2^n + n^2 + 2 = \Omega(n^2)$

✱ $f(n) = 10n^2 + 4n + 2 = \Omega(n)$

✱ $f(n) = 3n + 1 = \Omega(1)$



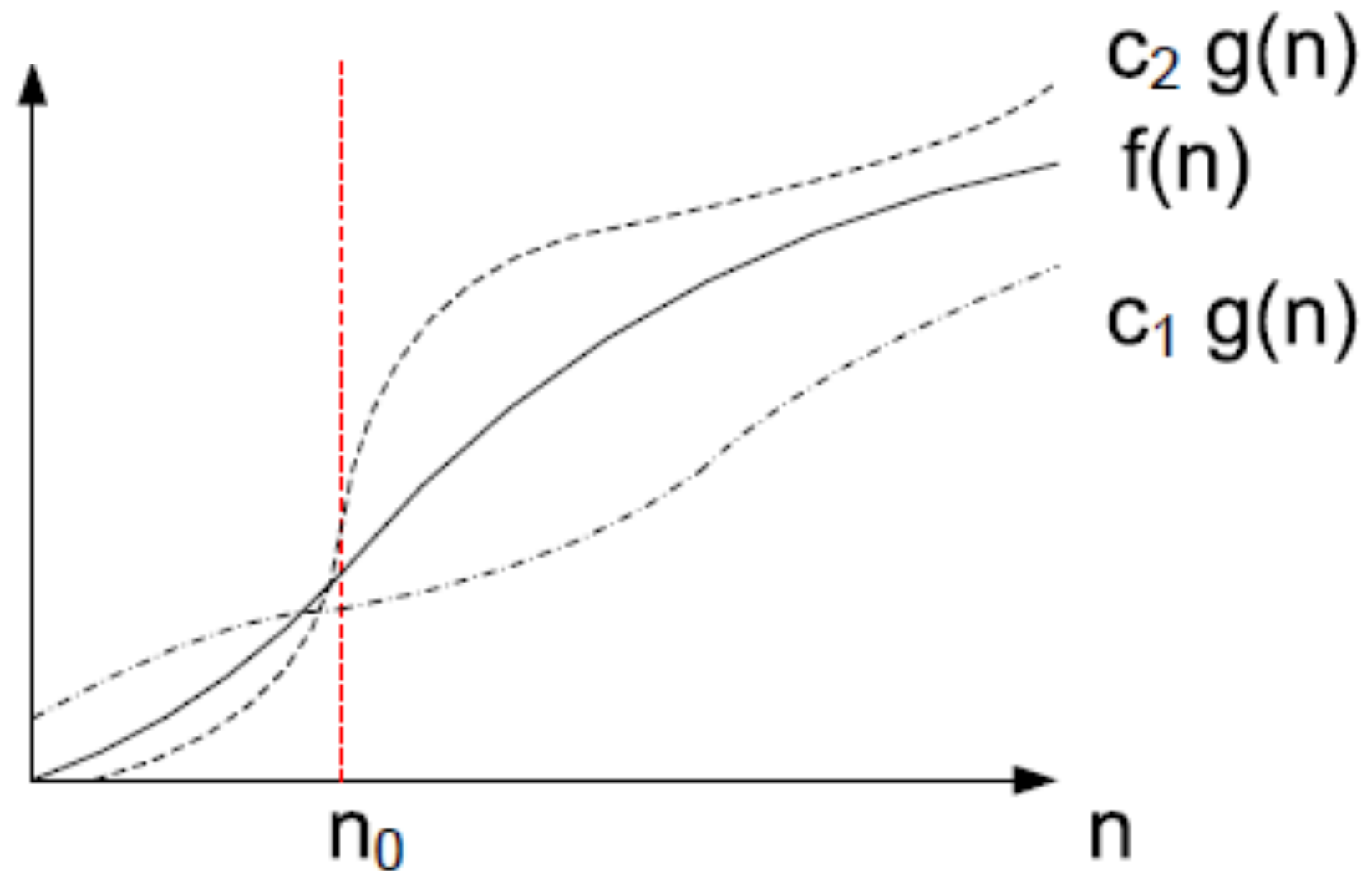
有很多很多的函數 $g(n)$ 可以滿足
 $f(n) = \Omega(g(n))$
只是大多我們都會使 $g(n)$ 去代表
一個 $f(n)$ 的下限
即 $g(n)$ 要盡可能的大，且逼近 $f(n)$

Theta Θ

* Definition [Theta]:

- * Read as “f of n is theta of g of n”
- * $f(n) = \Theta(g(n))$ iff there exist positive constants c_1 , c_2 and n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all n , $n \geq n_0$.
- * Lower bound and upper bound on $f(n)$
- * The theta notation is more precise than both “big oh” and omega notations.

Theta Θ



Examples of Theta

✿ $3n+2 = \Theta(n)$ as $3n+2 \geq 3n$ for all $n \geq 2$, and $3n+2 \leq 4n$ for all $n \geq 2$.

✿ $10n^2+4n+2 = \Theta(n^2)$

✿ $6 \cdot 2^n + n^2 = \Theta(2^n)$

✿ $10n^2+4n+2 \neq \Theta(1)$

✿ $g(n)$ should be the upper and lower bound on $f(n)$

✱ Notice :

✱ The coefficients in all the $g(n)$'s used in the preceding three example have been 1

✱ We shall almost never find ourselves saying that

✱ $3n+2=O(3n)$, $10=O(100)$, $10n^2+4n+2= \Omega(5n^2)$ or
 $6*2^n+n^2= \Theta(4*2^n)$

✱ Even though each of these statement is true.

Hands-on

✱ Show that the following equalities are correct:

✱ $5n^2 - 6n = \Theta(n^2)$

✱ Show that the following equalities are incorrect:

✱ $n^2 \log n = \Theta(n^2)$

Prove

- * $c_1 n^2 \leq n^2 \log n \leq c_2 n^2$ for all n , $n \geq n_0$
 - * $c_1 n^2 \leq n^2 \log n \rightarrow c_1 \leq \log n$
 - * $n^2 \log n \leq c_2 n^2 \rightarrow c_2 \geq \log n \rightarrow n \leq 2^{c_2}$ (c_2 is a constant)
 - * for all n , $n \geq n_0$, but $n \leq 2^{c_2}$
- * **Contradiction!**
- * $\rightarrow n^2 \log n = \Theta(n^2)$ is incorrect

Real Cases of Theta

- ✿ Asymptotic complexity of matrix addition

- ✿ $\Theta(\text{rows} * \text{cols})$

- ✿ Table 1.6

- ✿ Asymptotic complexity of permutation

- ✿ $\Theta(n (n!))$

- ✿ Example 1.8

- ✿ Asymptotic complexity of binary search

- ✿ worst case: $\Theta(\log n)$

- ✿ best case: $\Theta(1)$

- ✿ Example 1.9

- ✿ Asymptotic complexity of magic square: $\Theta(n^2)$

- ✿ Example 1.20

Example magic square

15	8	1	24	17
16	14	7	5	23
22	20	13	6	4
3	21	19	12	10
9	2	25	18	11

void Magic (const int n)

//產生一個大小為 n 的魔術方陣，其中 n 為奇數 {

const int MaxSize = 51; // 最大方陣大小

int square [MaxSize][MaxSize], k, l;

// 確定 n 的正確性

if ((n > MaxSize) || (n < 1))

throw "Error!..n out of range " ;

else if (!(n%2)) throw "Error!..n is even \n" ;

// n 是奇數，可以用 Coxeter 規則

for (int i = 0; i < n; i++) // 將方陣起始為0

fill(square[i], square[i] + n, 0) ; // STL演算法

square[0][(n-1)/2] = 1; // 第一列的中間位置

// i 跟 j 是目前的位置

int key = 2; i = 0; int j = (n-1)/2;

while (key <= n*n) {

// 向左上移動

if (i-1 < 0) k = n-1; else k = i-1;

if (j-1 < 0) l = n-1; else l = j-1;

if (square[k][l] i = (i+1)%n; // square已經被填滿，往下移動

else { // square[k][l]未被填滿

i = k; j = l;

}

square[l][j] = key;

key++;

} // while結束

因為要填入 $n*n$
Step< n^2-1

// 輸出魔術方陣

cout << "magic square of size " << n << endl;

for (i = 0; i < n; i++) {

for (j = 0; j < n; j++)

copy(square[i], square[i] + n,

ostream_iterator<int>(cout, " "));

cout << endl;

}

}

Practical Complexities

- * Assume the program P has complexity $\Theta(n)$ and program Q is of complexity $\Theta(n^2)$
- * We can assert that program P is faster than program Q for sufficiently large n

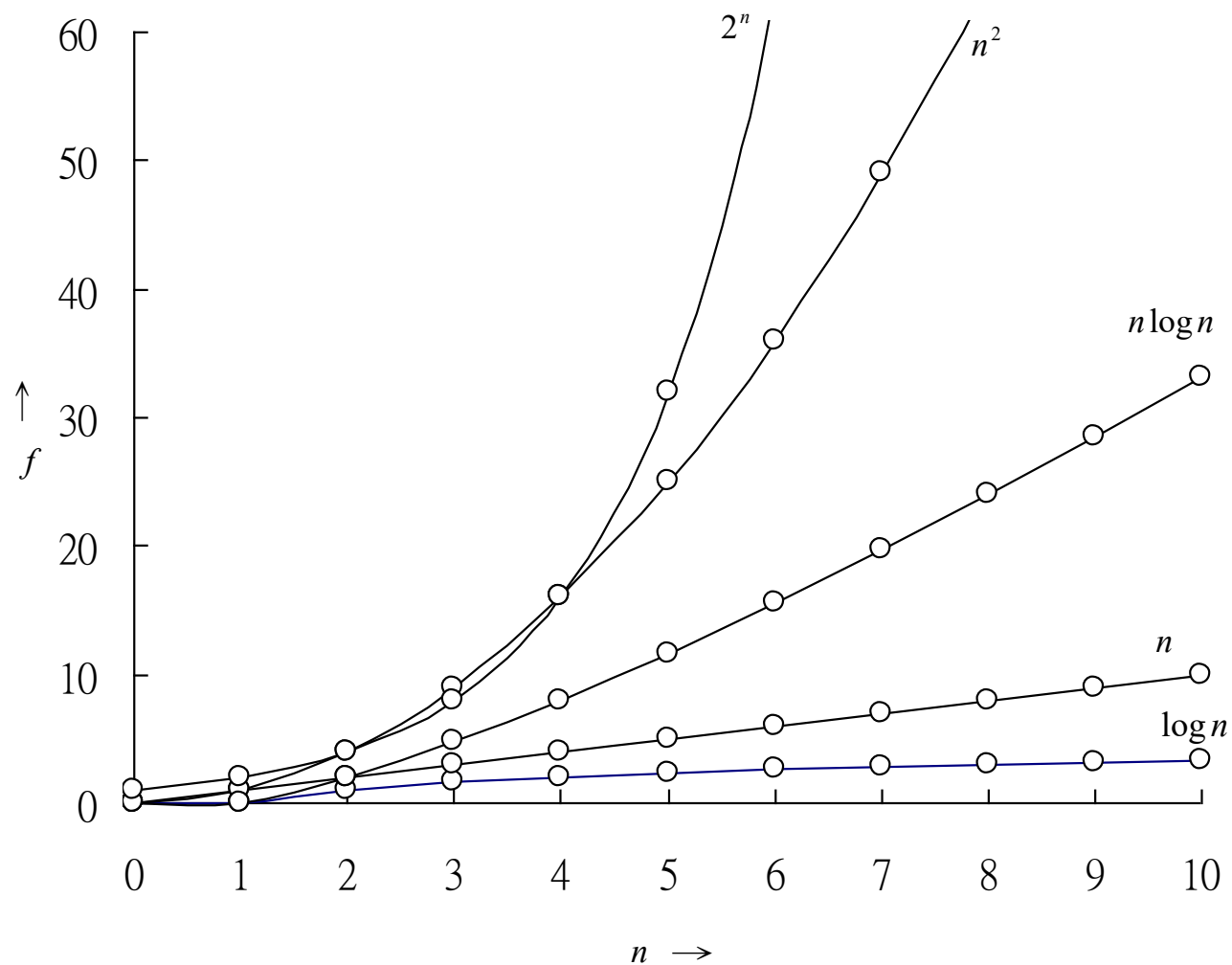
Table 1.7

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65,536
5	32	160	1024	32,768	4,294,967,296

The utility of programs with exponential complexity is limited to small n (n=40)

If a program need 2^n steps for execution, then when $n=40$, the number of steps needed is approximately 1.1×10^{12} ; a computer performing 1 billion steps per second, this would require about 18.3 minutes. $n=50 \rightarrow 13$ days!!!

Plot of function value



The function 2^n grows very rapidly with n

	$f(n)$						
n	n	$n \log_2 n$	n^2	n^3	n^4	n^{10}	2^n
10	.01 μ s	.03 μ s	.1 μ s	1 μ s	10 μ s	10s	1 μ s
20	.02 μ s	.09 μ s	.4 μ s	8 μ s	160 μ s	2.84h	1ms
30	.03 μ s	.15 μ s	.9 μ s	27 μ s	810 μ s	6.83d	1s
40	.04 μ s	.21 μ s	1.6 μ s	64 μ s	2.56ms	121d	13d
50	.05 μ s	.28 μ s	2.5 μ s	125 μ s	6.25ms	3.1y	$4 \cdot 10^{13}$ y
100	.10 μ s	.66 μ s	10 μ s	1ms	100ms	3171y	$32 \cdot 10^{283}$ y
10^3	1 μ s	9.96 μ s	1 ms	1s	115.7d	$3.17 \cdot 10^{13}$ y	
10^4	10 μ s	130 μ s	100 ms	11.57d	3171y	$3.17 \cdot 10^{23}$ y	
10^5	100 μ s	1.66 ms	10s	31.71y	$3.17 \cdot 10^7$ y	$3.17 \cdot 10^{33}$ y	
10^6	1ms	19.92ms				$3.17 \cdot 10^{43}$ y	

μ s = 百萬分之一秒 = 10^{-6} 秒；ms = 千分之一秒 = 10^{-3} 秒
s = 秒；m = 分鐘；h = 小時；d = 日；y = 年；

Times on a 1-billion-steps-per-second computer

Even if one could build a computer capable of executing 10^{12} instructions per second.

The computing time in this table would decrease factor of 1000

When $n=100$, it would take 3.17 years to execute n^{10} instructions.

Performance Measurement

- ✿ It is concerned with obtaining the actual space and time requirement of a program.
- ✿ As computers are continuously getting faster, your time will, most likely, be much smaller than those report in this book.
- ✿ To obtain the computing time (run time) of a program, we are a clocking function
 - ✿ In C++, we should include “time.h”

Real Time Complexity

```
#include "stdafx.h"
#include <iostream>
#include "time.h"
using namespace std;
int _tmain(int argc, _TCHAR* argv[])
{
```

A screenshot of a Windows command prompt window. The title bar shows the path 'c:\Users\catpin\Desktop\CalPr...'. The window contains the following text: 'start clock tick =53', 'end clock tick =5637', 'total time =5.584second', and '請按任意鍵繼續'.

程式開始跑之前

```
clock_t start_time, end_time;
```

```
float total_time = 0;
```

```
start_time = clock();
```

```
//Returns the number of clock ticks elapsed since the program was launched.
```

```
//clock tick: CPU時鐘節拍
```

```
//--program--//
```

```
double x=0;
```

```
for(double i=0;i<1000000000;i++)
```

```
x=x+i;
```

```
//-----//
```

```
end_time = clock();
```

```
total_time = (float)(end_time - start_time)/CLOCKS_PER_SEC;
```

```
cout << "start clock tick ="<<start_time<<endl;
```

```
cout << "end clock tick ="<<end_time<<endl;
```

```
cout << "total time ="<<total_time<<"second"<<endl;
```

```
system("pause");
```

```
return 0;
```

```
}
```

程式跑完之後

Real Time Complexity

```
#include "stdafx.h"
#include <iostream>
#include "time.h"
using namespace std;
int _tmain(int argc, _TCHAR* argv[])
{
```

程式開始跑之前



```
    time_t start_time, end_time;
```

```
    double total_time = 0;
```

```
    start_time = time(NULL); // 傳回從January 1, 1970到現在的秒數
```

```
    //--program--//
```

```
    double x=0;
```

```
    for(double i=0;i<10000000000;i++)
```

```
    x=x+i;
```

程式跑完之後



```
    //-----//
```

```
    end_time = time(NULL);
```

```
    total_time = difftime(end_time, start_time);
```

```
    cout << "start time =" << start_time << endl;
```

```
    cout << "end time =" << end_time << endl;
```

```
    cout << "total time =" << total_time << "second" << endl;
```

```
    system("pause");
```

```
    return 0;
```

```
}
```

- ✿ 在PC上準確的測量執行時間是不太可能的OS中要做的事情太多了，尤其是在系統負載很高時，時間的計算會更不準確。
- ✿ 且這樣的計算方式是近似值，如果要準確的估算到千分之1秒以下得靠其他的方法
 - ✿ 就跟量測一張紙的厚度是一樣的
 - ✿ 重複多次再除以次數

1st LeetCode HW

- ✿ Kth Smallest Element in a Sorted Matrix
- ✿ <https://leetcode.com/problems/kth-smallest-element-in-a-sorted-matrix/description/>
- ✿ **Deadline: 9/26(下週五) pm 5:00前**

All My Submissions

Time Submitted	Question	Status	Runtime	Language
2 weeks ago	Remove Element	Accepted	6 ms	cpp
2 weeks ago	Remove Element	Compile Error	N/A	cpp
2 weeks ago	Remove Element	Compile Error	N/A	cpp
2 weeks ago	Remove Element	Compile Error	N/A	cpp
2 weeks ago	The K Weakest Rows in a Matrix	Accepted	33 ms	cpp
2 weeks ago	The K Weakest Rows in a Matrix	Accepted	24 ms	cpp
2 weeks ago	The K Weakest Rows in a Matrix	Accepted	11 ms	cpp

Newer

Older



Yi-Ping Chao

catpin

Rank 3,226,237

Sports, Movie, Camping, Cooking

Taiwan

Community Stats

Views 0
Last week 0

Solution 0
Last week 0

Discuss 0
Last week 0

Reputation 0
Last week 0

Languages

C++

2 problems solved

Skills

Advanced

Solved Problems



Easy 2 / 595 Not enough data
Medium 0 / 1293 Not enough data
Hard 0 / 534 Not enough data

Badges

0

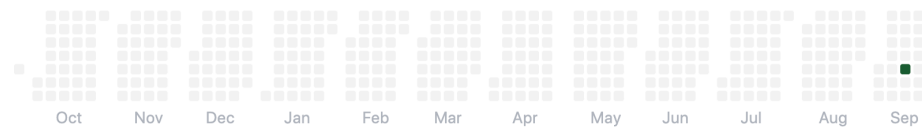


Locked Badge

Oct LeetCode Challenge

7 submissions in the last year

Total active days: 1 Max streak: 1



Recent AC

Solutions

Discuss

Remove Element

14 days ago

The K Weakest Rows in a Matrix

14 days ago

WORD內必須包含上述畫面與程式碼

Homework 2

* Exercise

* P 69 1.7.3 (a)(b)

* P 71 1.7.8 (a)(i)(l)

* P 72 1.7.9 (b)

* P 73 1.7.10 (需要設計實驗、實際運行，並用excel畫出表跟時間圖)

* P 73 1.7.12 (需要設計實驗、實際運行，並用excel畫出表跟時間圖)

* Deadline: 10/3(五) pm 5:00

* 一律用電腦打字，存成word後，上傳EL system



資料結構與演算法 I

(Data Structure and Algorithms I)

2025 fall

2025/09/19

Abstract Data Type & Chapter 2

Data Abstraction & Encapsulation

✿ Chapter 1.3

✿ Abstraction(抽象化)

✿ It is the separation between specification of a data object and its implementation

✿ 將一個資料物件規格及其內部實作分開

✿ Encapsulation(封裝化)

✿ Data encapsulation or information hiding is the concealing of the implementation details of a data object from the outside world

✿ 資料封裝或資訊隱藏是將資料物件的內部程式碼細節與外界隔絕

Example-DVD player

- ✱ All our interactions with a DVD players—made through buttons on the remote, such as PLAY, STOP...
- ✱ The DVD player is packaged so we cannot directly interact with the circuit inside
 - The **internal representation** of the DVD player is **hidden** from the user
 - DVD播放器的內部電路架構都隱藏起來不讓使用者看到
 - It is the concept of “*encapsulation*”



- ✿ The instruction manual that accompanies the DVD player tells us *what* it is supposed to do if we press a particular button. But not *how* this function is implemented inside the DVD player
- ✿ 反正就是可以動，可以播放，但是不需要也無法知道如何控制DVD播放器工作原理及其電路實作方法
- ✿ The instruction manual makes a clear distinction between *what* DVD player does and *how* it does it
- ✿ This distinction is the principle of “*abstraction*”
- ✿ The same principles may be applied to the packaging or organizing of data in a computer program
- ✿ *Data abstraction & data encapsulation*

Data Type

- ✿ Fundamental data type of C++

- ✿ char, int, float, double, short, long, signed, unsigned

- ✿ Type derived from the fundamental data types:

- ✿ pointer, reference

- ✿ Grouping data types

- ✿ Arrays

- ✿ Arrays are collections of elements of the same basic data type

- ✿ structs

- ✿ classes

- ✿ They are collections of elements whose data types need not be the same

Data Type

* Definition

- * A data type is a collection of objects and a set of operations that act on those objects.

- * 一組性質相同的資料及其操作

- * 一組物件集合以及定義在此集合上的一些操作

* 高階語言：

- * 變數、常數、運算式都有其各自取值範圍

- * 型別：說明變數或運算式的取值範圍及其操作

- * For example

- * Data type **INT** consists of the objects {0, +1, -1, +2, -2..., MAXINT, MININT}

- * Operators : +, -, *, /, =, !=

Abstract Data Type (ADT)

✿ Definition:

- ✿ An Abstract data type (ADT) 抽象資料型態 is a data type that is organized in such a way that the specification of the objects and the specifications of the operations on the objects is separated from the representation of the objects and the implementation of the operations
- ✿ 一種組織過的資料型態，由於組織的方式使我們得以將物件及物件內之運算操作兩者之規格分離，並且也可以分開物件表示方式與運算之實作方法

Abstract Data Type (ADT)

- ✿ 資料型別是根據值的不同而進行劃分，在高階語言(C, C++, C#, Java...)每種變數、常數、運算式都有各自的取值範圍，型別就是用以說明變數或運算式的取值範圍，以及其所能進行的操作
- ✿ 為啥當初設計電腦語言的人會想到要考慮資料型別？
- ✿ 因為需求不同
 - ✿ 大家都需要房子住，有人要住100坪、有人沒錢只能住五坪小套房，甚至是膠囊公寓，以滿足住的需求
- ✿ 電腦記憶體容量有限
- ✿ 計算 $1+1=2$ 或 $5+8=13$ 應該用char即可，應該不用規劃long的大小

✿ 根據C語言按照取值不同，資料型態可分為兩種

✿ 原子型態：表示不可再分解的基本型別：整數、浮點數、字元

✿ 結構型別：由多種型別組成，是可以再分解。如整數陣列、結構

✿ 變數宣告 `int a, b;` 就表示給變數a,b時，不能超過int的取值範圍，變數之間的運算只能是int型別所允許的運算

✿ 不同電腦會有不同的硬體系統，因此高階語言必須透過編譯器轉化為低階的組合語言或機器語言型別

✿ 而高階程式設計者根本不在乎最後程式會在何種電腦上運行，也不想知道CPU為了運算 $1+2$ 而進行了多少次運算及傳輸，高階程式設計師只在乎高階程式語言都需要用到整數運算、浮點數運算等操作→似乎要將其抽象出來

✿ 抽象：

- ✿ 取出事物具有普遍性的本質，抽出問題的特徵而忽略非本質的細節
- ✿ 是一種問題思考的方式，隱藏了繁雜的細節，只保留了實現目標所必要的資訊。

✿ ADT 抽象資料型別

- ✿ 指一個數學模型、以及定義在該模型上的一組操作
 - ✿ 在Mac, windows, mobile, Linux都擁有整數型別，也需要整數的運算，那整數就是一種抽象資料型別
 - ✿ 因為其實現及呈現方式不一定相同，但其定義之數學特性相同，對於電腦程式開發者來說，整數都是相同的
 - ✿ 程式開發者在設計軟體程式中，自行定義的資料型態也可視為一種抽象資料型別, Ex: Point

- ✳ 抽象資料型別還包括定義在該模型下的一組操作

- ✳ Mario

- ✳ Walk forward, backward

- ✳ Jump

- ✳ Fire

- ✳ Press fire+walk → run

- ✳ 根據實際需求，可再增加新的操作

- ✳ Fly

- ✳ Ride Yoshi?

} operations

- ✳ 抽象資料型別展現了程式設計中問題分解、抽象、以及資訊隱藏特性

Example of ADT

ADT *NaturalNumber* is

objects: An ordered sub-range of the integers starting at zero and ending at the maximum integer (MAXINT) on the computer.

functions:

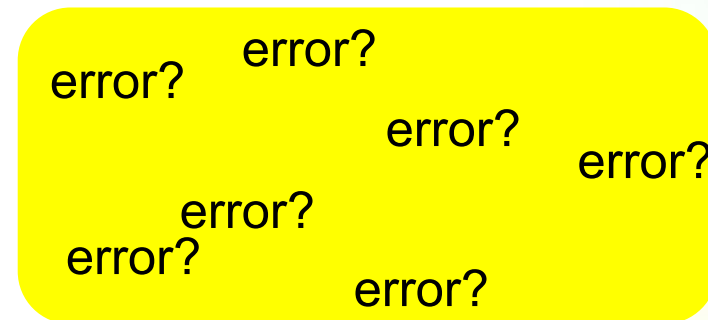
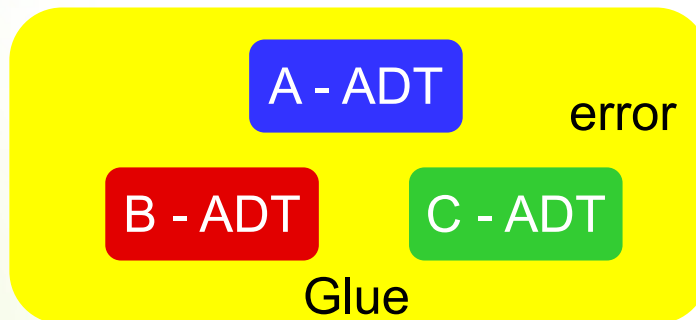
for all x, y belong to *NaturalNumber*; TRUE, FALSE belong to *Boolean* and where $+$, $-$, $<$, $==$, and $=$ are the usual integer operations

<i>Zero(): NaturalNumber</i>	::=	0
<i>IsZero(x): Boolean</i>	::=	if (x == 0) <i>IsZero</i> = TRUE else <i>IsZero</i> = FALSE
<i>Add(x, y): NaturalNumber</i>	::=	if (x+y <= MAXINT) <i>Add</i> = x + y else <i>Add</i> = MAXINT
<i>Equal(x, y): Boolean</i>	::=	if (x == y) <i>Equal</i> = TRUE else <i>Equal</i> = FALSE
<i>Successor(x): NaturalNumber</i>	::=	if (x == MAXINT) <i>Successor</i> = x else <i>Successor</i> = x + 1
<i>Substract(x, y): NaturalNumber</i>	::=	if (x < y) <i>Substract</i> = 0 else <i>Substract</i> = x - y

end *NaturalNumber*

Benefits of ADT

- ✿ Simplification of software development
 - ✿ Scenario 1- A team for four programmers
 - ✿ Scenario 2- A single programmer
- ✿ Testing & debugging



- ✿ Reusability
- ✿ Modifications to the representation of a data type

2.1.1 C++ Class

- ✳ C++ provides the *class* to support the distinction between specification and implementation and to hide the implementation of the ADT from its users.

- ✳ The C++ class consists of four components:

- ✳ A class name

- ✳ Data numbers

- ✳ i, j, k

- ✳ Member functions

- ✳ Get(); Set();

- ✳ Levels of program access

- ✳ public

- ✳ private

- ✳ protected

```
#ifndef RECTANGLE_H
```

```
#define RECTANGLE_H
```

```
class Rectangle{
```

```
public:
```

```
    Rectangle();
```

```
    ~Rectangle();
```

```
    int GetHeight();
```

```
    int GetWidth();
```

```
private:
```

```
    int xLow, yLow, height, width;};
```

```
#endif
```

公用

私用

* public

- * any public data member (member function) can be accessed (invoked) from anywhere in the program

* private

- * a private data member (member function) can be only accessed (invoked) from within its class or by a function or a class that is declared to be a friend.

* protected

- * A protected data member (member function) can only be accessed (invoked) from within its class or its subclasses or by a friend.

```
// In the source file Rectangle.cpp  
#include "Rectangle.h"
```

```
// 用"Rectangle::"來指明GetHeight()與GetWidth()是Rectangle的成員函式  
// 成員函式是在它們所屬類別定義的外部實作的
```

```
int Rectangle::GetHeight() {return height;}  
int Rectangle::GetWidth() {return width;}  

```

```
// In a source file main.cpp
```

```
#include <iostream>
```

```
#include "Rectangle.h"
```

```
main() {
```

```
    Rectangle r, s ;    // 物件 r跟 s屬於類別Rectangle
```

```
    Rectangle *t = &s ; // t 是類別物件s的指標
```

```
    // 使用 "." 來存取類別物件的成員。
```

```
    // 使用 "->" 透過指標來存取類別物件的成員。
```

```
    If ( r.GetHigh ( ) * r.GetWidth ( ) > t->GetHeight ( ) * t->GetWidth ( ) )
```

```
        cout << "r";
```

```
    else cout << "s";
```

```
    cout << "has the greater area " << endl;
```

```
}
```

2.1.4 Special Class Operations

✿ Constructors

- ✿ a member function which initialize data members of an object
- ✿ if it is provided for a class, it is automatically executed when an object of that class is created.
- ✿ If not, memory is allocated for the data members of a class object, but the data member are not initialized **Program 2.1**

✿ Destructors

- ✿ Destructors are automatically invoked when a class object goes out of scope or when a class object is deleted

✿ Operator overloading

- ✿ **Program 2.6, 2.7**